

Empirical AutoML on Tabular Data

AutoML – Assignment 1

Leiden University

The objective of this assignment is to compare three approaches to hyperparameter optimisation: Random Search, Sequential Model-Based Optimisation (SMBO), and Hyperband. Optimise the same random forest under a shared experimental protocol, using an untuned forest as a baseline. On the largest dataset, also evaluate a pre-trained tabular foundation model and compare it with the baseline and forests tuned by each optimiser.

You will study predictive performance, optimisation progress, computational cost, and variation across repeated runs. Choose and justify measures of performance and compute that suit your research questions, considering what they capture and their limitations.

1 Datasets

Use five OpenML classification datasets with strictly increasing row counts. The supplied suite in `data_loading.py` uses version-pinned OpenML identifiers [2].

Dataset key	OpenML ID	Rows	Predictors	Classes
<code>breast-w</code>	15	699	9	2
<code>credit-g</code>	31	1,000	20	2
<code>phoneme</code>	1489	5,404	5	2
<code>electricity</code>	151	45,312	8	2
<code>covertypes</code>	1596	581,012	54	7

Increasing size does not imply increasing prediction difficulty: features, class distributions, and tasks also differ. You may replace datasets with version-pinned OpenML alternatives while retaining strictly increasing row counts. For each replacement, explain your choice and report the prediction target, OpenML ID, and numbers of rows, predictors, and classes. The foundation-model experiment must use the largest selected dataset.

Choose and justify an evaluation protocol, explaining your use of training, validation, and test data, including preprocessing and final refitting. Explain how your protocol prevents test-data leakage and supports a fair comparison across methods. The supplied stratified holdout is a starting point. Other statistically sound protocols are welcome.

Set the comparison rules before running the main study, apply them consistently, and distinguish supplementary analyses.

The main forest study compares the untuned forest with forests tuned by Random Search, SMBO, and Hyperband on all five datasets. Use all rows, partitioned according to your protocol, and do not subsample or cap rows.

Hint: you do not have to compare TabPFN with other methods under the same logic of compute measurements. For example, a potential study may compare forests and a foundation model such as TabPFN at equal labelled-data budgets (Section 3.5). Keep selection within that budget and evaluate the complete held-out test set.

2 Shared optimisation problem

Use scikit-learn's `RandomForestClassifier` for all three optimisers, with a shared search space and validation objective. Explain the hyperparameters and ranges you choose, ensuring that all configurations are legal. The supplied search space is a starting point that you may adapt with justification.

Choose and justify a *predictive metric* to measure prediction quality. For each candidate configuration, evaluate the trained forest on validation data. The *validation objective* is the value the optimiser uses to compare these candidates. It may be the metric itself or a value derived from it. Explain this relationship and whether larger or smaller objective values are better. Apply that direction consistently across optimisers, and use consistent metric definitions for all methods and final test results. The optional `random_forest.py` provides scoring placeholders.

For this assignment, use the number of trees as the resource allocated by Hyperband. Choose a common maximum tree count: Random Search and SMBO train every candidate forest with this many trees, while Hyperband can evaluate candidates with fewer trees and allocate more to promising ones. For final testing, train each method's selected configuration and the untuned baseline with the common maximum. This compares the settings found for forests of the same size while allowing Hyperband to save work during search. Tree count is therefore controlled by resource allocation rather than searched alongside the other hyperparameters.

Choose and justify how you measure computational effort and use it to compare methods fairly. You may examine computational cost and how efficiently solvers find good configurations. Choose measures that support your questions and explain what your measurements include, any assumptions or differences that affect the comparison, and the hardware used. Set the experimental limits before running the main study.

3 Methods

Implement or configure the methods using your own code or suitable packages. Explain how each works at a high level and describe your main settings.

For all forest methods, refit preprocessing and the final model on all non-test data within the study's declared labelled-data budget before final testing.

3.1 Untuned random forest baseline

Use scikit-learn's default hyperparameters, with the tree count set to the common maximum in Section 2. Report the baseline alongside the optimised methods.

3.2 Random Search

Random Search samples configurations randomly from the shared search space and selects the best according to the validation objective [1]. Explain your sampling choices and evaluation budget, and save enough information to analyse performance as the search progresses.

3.3 SMBO

SMBO builds a model from previous configuration evaluations and uses it to guide which settings to try next [4, 6].

3.4 Hyperband

Hyperband evaluates configurations in stages. At each stage, it keeps the better-performing configurations and evaluates them with more trees, keeping their other hyperparameter settings unchanged [5]. It runs several *brackets*, each starting with a fresh group of configurations: some start with many configurations and few trees per forest, others with fewer configurations and more trees. Justify your allocation schedule and how you select the final configuration.

3.5 Pre-trained tabular foundation model

Evaluate a pre-trained tabular foundation model on the largest dataset in your suite. TabPFN is one example [3], however other suitable models are welcome. Depending on the model, you may provide labelled examples as context for prediction (*in-context learning*), update its weights using task data (*fine-tuning*), or combine these approaches.

Choose and justify a direction to explore, including the model, how you use it, and the data available to it. You may use a justified subset of the data available for training or adaptation. Evaluate on the same complete test set as the forest methods, keeping it separate from all training, adaptation, and model-selection decisions.

Compare your approach with the untuned forest and the forests tuned by Random Search, SMBO, and Hyperband in predictive performance and computational effort. You may use a different comparison design or presentation from that used among forest optimisers. Justify your choices and fairness assumptions, explaining how differences in data use and compute affect your conclusions.

4 Code organisation and reproducibility

The supplied student code is optional: complete or adapt it, or build your own pipeline. The same experimental requirements apply. Code structure is your choice.

Organise your code so that data preparation, methods, evaluation, and result generation are easy to locate. Use clear names and concise documentation, and make experimental settings easy to find and change.

Supply all scripts and a README with installation instructions and commands for pipeline checks and reproducing experiments, tables, and figures.

Before the main experiments, decide what evidence to retain for each method so that you can analyse final performance, optimisation progress, and computational effort.

Check the pipeline on a small example before starting the main experiments. Check foundation-model access and expected runtime early.

5 Experiments and report

State the specific questions your study investigates and explain how your experimental design provides evidence to answer them.

Use at least three experimental seeds for each method on each dataset in the main forest study. Explain what varies between runs and justify the number of runs in supplementary studies.

Your report should address:

1. How do the optimisers compare with the untuned forest across the five datasets?
2. How efficiently do they find good configurations?

3. How does the foundation-model approach compare with all forest methods in predictive performance and computational effort?

Choose and justify how you aggregate repeated runs and show variation. Use figures or tables that clearly support your comparisons, explaining the measures and assumptions used. Distinguish repeated runs from experiments with different settings. Discuss the main limitations and what your results support.

6 Submission and assessment

Write a scientific report of at most four pages, excluding references, using the provided LaTeX template. Keep the template formatting unchanged. Each additional page of report body incurs a 1.0-point deduction. These assignment-specific rules take precedence over general reporting advice.

Submit through Brightspace:

- The report as a separate PDF for plagiarism checking.
- Your code as a ZIP, including the README, dependencies, and all scripts needed to reproduce the experiments, tables, and figures.

Do not include downloaded datasets, model checkpoints, caches, or virtual environments. The deadline, group rules, and late-submission policy are published on Brightspace.

Assessment considers correct methods, sound experimental comparisons, justified choices, interpretation of results, and clear, reproducible code and reporting. A particular winning method or score is not expected.

Passing requires a report, code with reproduction instructions, a working small pipeline check, and an interpretable experimental comparison of multiple required methods. Completeness across datasets, methods, and repeated runs is assessed through the grading criteria.

References

- [1] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [2] Matthias Feurer, Jan N. van Rijn, Arlind Kadra, Pieter Gijsbers, Neeratyoy Mallik, Sahithya Ravi, Andreas Müller, Joaquin Vanschoren, and Frank Hutter. Openml-python: an extensible python API for openml. *Journal of Machine Learning Research*, 22:100:1–100:5, 2021.
- [3] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeyer, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8044):319–326, 2025.
- [4] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *Learning and Intelligent Optimization - International Conference, LION 5*, volume 6683, pages 507–523. Springer, 2011.
- [5] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18:185:1–185:52, 2018.
- [6] Jasper Snoek, Hugo Larochelle, and Ryan Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012.